
UNIT 3: DATA TRANSFORMATION

Structure	Page No.
3.0 Introduction	63
3.1 Objectives	63
3.2 Discretisation and Binning	64
3.2.1 Conceptual Framework: Why Discretise?	64
3.2.2 Method 1: Equal-Width Binning with <code>pd.cut()</code>	65
3.2.3 Method 2: Equal-Frequency Binning (Quantile Binning) with <code>pd.qcut()</code>	67
3.2.4 Comparison of <code>pd.cut()</code> and <code>pd.qcut()</code>	68
3.3 Detecting and Filtering Outliers	69
3.3.1 Visualisation: Identifying Outliers with Box Plots	69
3.3.2 Method Selection: Z-score versus IQR	70
3.3.3 Z-Score Method (For Normal Data)	70
3.3.4 IQR Method (For Skewed Data)	71
3.4 Permutation and Random Sampling	75
3.4.1 Random Sampling without Replacement	76
3.4.2 Random Sampling with Replacement (Bootstrapping)	78
3.4.3 Permutation (Shuffling Data Order)	79
3.5 Computing Indicator/Dummy Variables	81
3.5.1 The Challenge of Categorical Data	81
3.5.2 The Principle of One-Hot Encoding	81
3.6 String manipulation and Regular Expressions	84
3.6.1 Vectorised String Object Methods in pandas	84
3.6.2 Structural Manipulations and Feature Creation	85
3.7 Summary	89
3.8 Solutions/Answers	89
3.9 Further Readings and References	90

3.0 INTRODUCTION

Welcome to the Unit on Data Transformation! As aspiring Data Scientists, you must recognise that raw data is seldom ready for direct consumption by analytical models. Data transformation is the crucial stage where we convert raw, inconsistent, or poorly structured data into an optimised, standardised format that facilitates efficient learning and prediction.

The primary goal of transformation is to adjust the data distribution, scale, or representation to meet the specific requirements of modeling algorithms, thereby improving model performance and interpretability. For instance, some algorithms assume the data follows a normal distribution, while others perform better when categorical variables are converted into numerical representations.

In this unit, we will systematically explore five foundational transformation techniques essential for manipulating both numerical and text data: handling continuous values through binning, managing extreme values (outliers), generating representative samples, preparing categorical variables, and cleaning text using powerful string operations and regular expressions.

3.1 OBJECTIVES

After successfully completing this unit, you will be able to:

- Define the necessity of data discretisation and apply Python functions for both equal-width and equal-frequency binning.
- Analyse data distribution characteristics to appropriately select and implement methods (Z-score or IQR) for detecting and treating statistical outliers.
- Perform random sampling to select representative data subsets and execute permutation (shuffling) to remove potential order bias in datasets.
- Create Binary Indicator / Dummy Variables from nominal categorical features while correctly addressing the multicollinearity problem.
- Utilise Pandas String Object methods and apply the principles of Regular Expressions for cleaning, validating, and extracting features from complex text fields.

3.2 DISCRETISATION AND BINNING

Discretisation is the process of converting continuous numeric data into discrete categories to facilitate certain modelling techniques. This section describes the process in detail.

3.2.1 Conceptual Framework: Why Discretise?

When working with continuous numerical variables, like age, income, or temperature, we often find that a model's performance or the analysis's interpretability can be improved by grouping these values into a finite number of discrete categories or bins. This process is called Discretisation or Binning.

For example, instead of using the exact Monthly_Salary (e.g., 50000, 68000, 150000), we may prefer to categorise it into 'Low Salary', 'Medium Salary', and 'High Salary'. We may group 'Age' into 5 or 10 age ranges (e.g., 0-10, 11-20, etc.).

This process offers several benefits:

- **Data Simplification:** By grouping values, we reduce the complexity of the dataset and make it easier to analyse and interpret.
- **Increased Model Stability:** Converting continuous features to categorical ones can sometimes reduce the impact of small fluctuations or noise in the data.
- **Optimising Tree-Based Models:** A critical application lies in preparation for decision tree algorithms (like Random Forests). When continuous features are discretised, the algorithm has fewer potential "cut-points" to evaluate during the training process, leading to reduced training time and often improved predictive performance.

Once the data points are assigned to bins, these bins can be treated as ordinal or nominal categorical data for downstream modeling.

Sample Datasets: To demonstrate all transformation techniques in this unit, we will use a small, synthetic dataset representing customer purchase behaviour.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Simulating Customer Purchase behavior
data = {
    'Customer_ID': range(1, 21),
    'Age': [18, 22, 25, 27, 30, 32, 35, 38, 40, 42, 45, 50, 52,
55, 58, 60, 62, 65, 68, 70],
    'Annual_Income': [15000, 18000, 25000, 27000, 29000, 31000,
33000, 35000, 38000, 40000,
```

```

42000, 45000, 47000, 49000, 50000, 55000,
70000, 85000, 120000, 200000],
'Spending_Score': [35, 40, 45, 42, 50, 48, 60, 58, 55, 52,
48, 46, 43, 41, 38, 30, 25, 20, 15, 10]
}

df = pd.DataFrame(data)
print("--- Original Data ---")
print(df)

```

Program 1: Program to display a sample data set**Output:**

```

--- Original Data ---
   Customer_ID  Age  Annual_Income  Spending_Score
0            1   18         15000             35
1            2   22         18000             40
2            3   25         25000             45
3            4   27         27000             42
4            5   30         29000             50
5            6   32         31000             48
6            7   35         33000             60
7            8   38         35000             58
8            9   40         38000             55
9           10   42         40000             52
10           11   45         42000             48
11           12   50         45000             46
12           13   52         47000             43
13           14   55         49000             41
14           15   58         50000             38
15           16   60         55000             30
16           17   62         70000             25
17           18   65         85000             20
18           19   68        120000             15
19           20   70        200000             10

```

Table 1: Sample Dataset**3.2.2 Method 1: Equal-Width Binning with pd.cut()**

In Equal-Width Binning, the range of the variable (calculated as Maximum value minus Minimum value) is divided into a predetermined number of intervals (N), where each interval has the exact same width. For example, if marks range from 40 to 100 (range of 60), and we choose $N=4$ bins, each bin will have a width of 15 ($60/4$).

We use the Pandas function `pd.cut()` for this method. Let us try an example using the above dataset.

```

#Include the CODE of Program 1, or else this program will give
an error
# Define the number of bins
num_bins = 5

# Apply equal-width binning using pd.cut()
# This divides the 18-70 range into 4 equally sized chunks
df['Age_Group'] = pd.cut(df['Age'], bins=num_bins)

print("\n--- Data after Equal-Width Binning ---")
print(df)
print("\nValue Counts for Bins:")
print(df['Age_Group'].value_counts())

```

Program 2: Code for Simple Equal-Width Segmentation on Age column

Program 2 shows the code for creating equal width segmentation of data of Age column. Please note that this code has to be appended after the code of Program 1 that contains the sample data. The program simply creates a new column named Age_Group in the dataframe using equal width binning. The last line of the code counts the frequency of Age_Group categories. The output of this code is shown in Table 2.

Output:

```

--- Data after Equal-Width Binning ---
Customer_ID  Age  Annual_Income  Spending_Score  Age_Group
0            1   18           15000           35  (17.948, 28.4]
1            2   22           18000           40  (17.948, 28.4]
2            3   25           25000           45  (17.948, 28.4]
3            4   27           27000           42  (17.948, 28.4]
4            5   30           29000           50   (28.4, 38.8]
5            6   32           31000           48   (28.4, 38.8]
6            7   35           33000           60   (28.4, 38.8]
7            8   38           35000           58   (28.4, 38.8]
8            9   40           38000           55  (38.8, 49.2]
9           10   42           40000           52  (38.8, 49.2]
10           11   45           42000           48  (38.8, 49.2]
11           12   50           45000           46  (49.2, 59.6]
12           13   52           47000           43  (49.2, 59.6]
13           14   55           49000           41  (49.2, 59.6]
14           15   58           50000           38  (49.2, 59.6]
15           16   60           55000           30  (59.6, 70.0]
16           17   62           70000           25  (59.6, 70.0]
17           18   65           85000           20  (59.6, 70.0]
18           19   68          120000           15  (59.6, 70.0]
19           20   70          200000           10  (59.6, 70.0]

Value Counts for Bins:
Age Group
(59.6, 70.0]      5
(17.948, 28.4]    4
(28.4, 38.8]      4
(49.2, 59.6]      4
(38.8, 49.2]      3
Name: count, dtype: int64

```

Notice how the bins have equal age ranges (width), but the number of customers (counts) in each bin are unequal (3, 4, and 5).

Custom Labels and Edges (Intermediate): Defining Meaningful Groups

`pd.cut()` is most powerful when you need to define specific, domain-based thresholds, such as defining salary brackets or age groups that conform to known standards.

The function allows you to explicitly define the bin edges and apply custom, descriptive labels. For example, to categorise 'Age_group' into meaningful tenure groups, domain knowledge is used to define the cut points: 0-25 years (Youth), 25-40 years (Adult), 40-60 years (Middle-Aged), and 60+ years (Senior).

```

#Include the CODE of Program 1, or else this program will give
an error
# Define custom boundaries (edges)
bins = [0, 25, 40, 60, 100]
# Define labels corresponding to the bins
labels = ['Youth', 'Adult', 'Middle-Aged', 'Senior']

df['Age_Group'] = pd.cut(df['Age'], bins=bins, labels=labels)

print("Tenure Group Distribution:")
print(df[['Age', 'Age_Group']].head(10))
print("\nCounts per Tenure Group:")
print(df[['Age_Group']].value_counts())

```

Program 3: Code for Defining Meaningful Age Groups

In program 3, first meaningful bins and their labels are created and then `pd.cut()` function with suitable parameters is used to create `Age_Group`. The first ten records of `Age` and `Age_Group` columns are displayed using the method `print(df[['Age', 'Age_Group']].head(10))`. Finally, the frequency count of values are displayed using `print(df[['Age_Group']].value_counts())` method. The following is the output of Program 3.

Output:

```
Tenure Group Distribution:
   Age  Age_Group
0   18     Youth
1   22     Youth
2   25     Youth
3   27     Adult
4   30     Adult
5   32     Adult
6   35     Adult
7   38     Adult
8   40     Adult
9   42  Middle-Aged

Counts per Tenure Group:
Age_Group
Middle-Aged    7
Adult          6
Senior         4
Youth          3
Name: count, dtype: int64
```

By defining explicit boundaries and labels, the resulting feature, 'Age_Group' has become highly interpretable and directly useful for comparative analysis (e.g., comparing age across tenure groups).

3.2.3 Method 2: Equal-Frequency Binning (Quantile Binning) with `pd.qcut()`

Equal-Frequency Binning, implemented using `pd.qcut()`, focuses on balancing the number of observations in each bin, rather than balancing the range. This method uses percentiles, or quantiles, to define the bin boundaries. For instance, if we request four bins (quartiles), the boundaries will be defined such that the 25th, 50th, 75th, and 100th percentiles split the data, resulting in approximately equal counts in each bin. This approach is particularly valuable when the underlying data distribution is highly skewed, preventing all or most data points from clustering into one or two large bins.

```
#Include the CODE of Program 1, or else this program will give
an error
# Apply equal-frequency binning using pd.qcut()
# We request 4 bins (quartiles)
df['Income_Quantile'] = pd.qcut(df['Annual_Income'], q=4,
labels=['Low', 'Medium', 'High', 'Very High'])
df[['Annual_Income', 'Income_Quantile']].head(10)

print("\n--- Data after Equal-Frequency (Quantile) Binning ---")
print(df[['Annual_Income', 'Income_Quantile']])
print("\nValue Counts for Quantile Bins:")
print(df['Income_Quantile'].value_counts())
```

Program 4: Code for Implementing Quantile Binning

The program 4 uses labels to create a new column named `Income_Quantile`. You may observe that the output shows equal frequency in each category.

Output:

```

--- Data after Equal-Frequency (Quantile) Binning ---
Annual_Income Income_Quantile
0          15000          Low
1          18000          Low
2          25000          Low
3          27000          Low
4          29000          Low
5          31000      Medium
6          33000      Medium
7          35000      Medium
8          38000      Medium
9          40000      Medium
10         42000          High
11         45000          High
12         47000          High
13         49000          High
14         50000          High
15         55000      Very High
16         70000      Very High
17         85000      Very High
18        120000      Very High
19        200000      Very High

Value Counts for Quantile Bins:
Income_Quantile
Low          5
Medium       5
High         5
Very High    5
Name: count, dtype: int64

```

The output confirms that the frequency of observations across the four bins is now much more balanced (5). The width of the intervals is no longer equal, but the representation is maximised, making this method preferable when dealing with variables that exhibit heavy skewness.

3.2.4 Comparison of `pd.cut()` and `pd.qcut()`

Let us look at the fundamental difference between these two powerful discretization tools:

Function	Basis of Division	Focus/Criteria	Best Used When...
<code>pd.cut()</code>	Equal Value Range (Equal Width)	The <i>distance</i> between the bin edges must be equal.	Domain knowledge dictates fixed ranges (e.g., age groups, scores out of 100).
<code>pd.qcut()</code>	Equal Observation Count (Equal Frequency)	The <i>number of data points</i> in each bin must be equal (or nearly equal).	Data is highly skewed, and you need to ensure sufficient samples in every bin for comparison.

Check Your Progress – 1

1. If you have a dataset of customer incomes that is heavily skewed (most people earn low income, a few earn extremely high income), which binning method (`pd.cut` or `pd.qcut`) would you choose to ensure equal representation of customers in each bin? Why?

.....

.....

.....

2. Explain the primary advantage of converting a continuous feature into discrete bins before training a Decision Tree model.

.....

3. A dataset ranges from 0 to 100. If you use `pd.cut` with `bins=5`, what is the exact width of each interval?

.....

3.3 DETECTING AND FILTERING OUTLIERS

Outliers are data points that deviate significantly from other observations. They represent rare occurrences, measurement errors, or genuine extreme values. If ignored, outliers can skew statistical averages (like the mean) and standard deviations, potentially leading to poorly generalised analytical models. Effective data transformation requires robust methods for detecting and managing these anomalies.

Outliers can arise from:

1. **Genuine Errors:** Data entry mistakes (e.g., typing 150000 instead of 15000) or sensor malfunctions.
2. **True Extremes:** A legitimate, rare observation (e.g., a CEO's salary in an average employee dataset).

The Danger of Outliers: Outliers have a disproportionate impact on statistical measures. They inflate the mean and dramatically increase the standard deviation. For models, they heavily bias results, especially in distance-based algorithms like Linear Regression, by pulling the regression line towards themselves.

3.3.1 Visualisation: Identifying Outliers with Box Plots

The essential first step in outlier management is visualisation. The **Box Plot (or Box-and-Whisker Plot)** is the standard tool for quickly identifying data spread and potential outliers. It provides a clear graphical representation, showing the median and quartiles. Any points that fall outside the defined "whiskers", determined by the Interquartile Range (IQR) rule, typically represent potential outliers

Let us introduce a dataset of annual income (in thousands) that contains one extremely high salary to simulate an outlier scenario. Program 4 shows the code of this program. Please note it uses *seaborn* library of Python, which is used for making the box plot on Annual Income. In addition, please note that this box plot has a title.

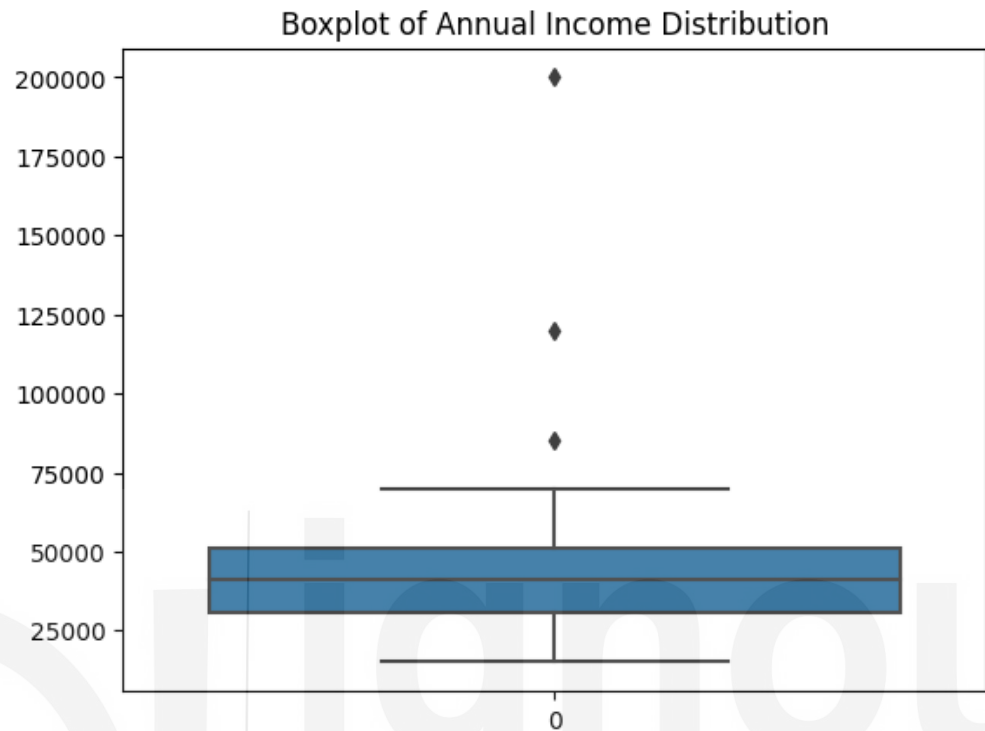
```
#Include the CODE of Program 1, or else this program will give
an error
import seaborn as sns
import matplotlib.pyplot as plt

# Visualising Outliers using a Box Plot (Conceptual)
# Visualization of Annual_Income distribution
sns.boxplot(df['Annual_Income'])
plt.title('Boxplot of Annual Income Distribution')
plt.show()
# Observations outside the whiskers are potential outliers.
```

Program 5: Visualising Outliers

In the following output, please note that three points shown as rhombus above the top whisker are potential outliers, as they exceed the value $1.5 \times IQR$.

Output:



3.3.2 Method Selection: Z-score versus IQR

The choice of numerical detection method depends critically on the underlying data distribution.

Z-Score Method

The Z-score method assumes that the data is normally (Gaussian) distributed. It measures how many standard deviations a data point deviates from the mean. Outliers are typically defined as values with a Z-score greater than a threshold, commonly ± 2 or ± 3 .

Interquartile Range (IQR) Method

The IQR method is preferred for skewed distributions because it relies on quartiles (percentiles), which are less sensitive to extreme values than the mean and standard deviation. The boundaries, often called Tukey's Fences, are calculated using the 25th percentile ($Q1$) and the 75th percentile ($Q3$). Data points falling outside $Q1 - 1.5 \times IQR$ or $Q3 + 1.5 \times IQR$ are flagged as outliers. This method is considered more robust than the Z-score for non-normal data.

3.3.3 Z-Score Method (For Normal Data)

For data that adheres to a normal curve, we define the allowed range based on the mean (μ) and the standard deviation (σ). The boundaries are calculated as:

$$\text{Upper Boundary} = \mu + 3 \times \sigma$$

$$\text{Lower Boundary} = \mu - 3 \times \sigma$$

We utilise NumPy and Pandas to calculate these limits and identify the aberrant data points. Once identified, outliers can be managed either by **trimming** (removing the observation) or **capping** (replacing the outlier with the boundary value).

```
#Include the CODE of Program 1, or else this program will give
an error
# Recalculate mean and std dev
df_outlier = df['Annual_Income']
mean = df_outlier.mean()
std_dev = df_outlier.std()
k = 3 # Standard threshold (3 standard deviations)

# Calculate boundaries
upper_limit_z = mean + k * std_dev
lower_limit_z = mean - k * std_dev

print(f"Mean: {mean:.2f}, Standard Deviation: {std_dev:.2f}")
print(f"Highest allowed (Z-score 3): {upper_limit_z:.2f}")
print(f"Lowest allowed (Z-score 3): {lower_limit_z:.2f}")

# Identifying the outlier row
outliers = df[(df_outlier>upper_limit_z) |
(df_outlier<lower_limit_z)]
print(f"\nOutliers detected (Z-Score): \n{outliers}")
```

Program6: Outlier Detection using Z-Score

The program first creates a separate containing only the *Annual_Income* column from the original data. This is followed by determination of mean and standard deviation of data of *df_outlier* dataframe. It then assumes the limiting constant *kas* 3, as shown in formula of Upper and Lower boundary, to compute the upper and lower boundary value from the mean of the data. Any data value above the upper boundary value and any data value lower than the lower boundary value is considered as potential outlier. The first three print statements print the mean, standard deviation, upper limit and lower limit using a format of 2 decimal places. Please note that these print statements use formatting *f*-string that allows printing of calculated value in a string. The rows from the dataframe *df*, which contain potential outliers of *Annual_Income*, are transferred to an object named *outliers*. Please note the use of logical operator OR (*|*) in this statement. Finally, all the rows having potential outlier in *Annual_Income* are printed.

Output:

```
Mean: 52700.00, Standard Deviation: 42281.14
Highest allowed (Z-score 3): 179543.42
Lowest allowed (Z-score 3): -74143.42

Outliers detected (Z-Score):
  Customer_ID  Age  Annual_Income  Spending_Score  Age_Group  Income_Quartile
19           20    70         200000             10    Senior        Very High
```

3.3.4 IQR Method (For Skewed Data)

The IQR method formalises the definition of outliers based on the quartiles. The Interquartile Range is calculated as: $IQR = Q3 - Q1$.

The IQR method is a statistically robust way to define precise boundaries for outliers, using quartiles:

1. Q1: First Quartile (25th percentile).
2. Q3: Third Quartile (75th percentile).

$$3. \text{ IQR} = Q3 - Q1.$$

Outliers are data points outside the fences(boundaries):

$$\text{Upper Limit} = Q3 + 1.5 \times \text{IQR}$$

$$\text{Lower Limit} = Q1 - 1.5 \times \text{IQR}$$

The following code finds the outliers using IQR.

```
#Include the CODE of Program 1, or else this program will give
an error
df_outlier = df['Annual_Income']
# Calculate Q1, Q3, and IQR
Q1 = df_outlier.quantile(0.25)
Q3 = df_outlier.quantile(0.75)
IQR = Q3 - Q1
multiplier = 1.5

# Calculate boundaries
upper_limit_iqr = Q3 + multiplier * IQR
lower_limit_iqr = Q1 - multiplier * IQR

print(f"Q1: {Q1:.2f}, Q3: {Q3:.2f}, IQR: {IQR:.2f}")
print(f"Highest allowed (IQR): {upper_limit_iqr:.2f}")
print(f"Lowest allowed (IQR): {lower_limit_iqr:.2f}")

# Identifying the outlier row
outliers = df[(df_outlier > upper_limit_iqr) |
(df_outlier < lower_limit_iqr)]
print(f"\nIQR Outliers: \n{outliers[['Customer_ID', 'Age',
'Annual_Income', 'Spending_Score']]}")
```

Program 7: Outlier Detection using IQR

Most of the statements used in Program 7 are similar to Program 6. The rows that contain the outlier values are shown in the output as given below.

Output:

```
Q1: 30500.00, Q3: 51250.00, IQR: 20750.00
Highest allowed (IQR): 82375.00
Lowest allowed (IQR): -625.00

IQR Outliers:
  Customer_ID  Age  Annual_Income  Spending_Score
17          18   65         85000             20
18          19   68        120000             15
19          20   70        200000             10
```

a. Strategies for Handling Outliers: Trimming and Capping

Once we have detected outliers in our dataset, the next question we face is how to handle them. This step is crucial because our choice directly affects the quality and reliability of our analysis. Essentially, we need to balance two priorities: reducing statistical noise while still preserving meaningful information. Let us look at two common strategies, trimming and capping, and discuss when each might be appropriate.

Trimming (Filtering Out)

Trimming, sometimes referred to as exclusion, means completely removing the observations that contain outlier values. You may find this to be the most straightforward method since it simply eliminates problematic data points. This approach works best when we are confident that the outliers result from errors, such as mistakes during data

entry, sensor malfunctions, or other anomalies that do not represent normal behavior in the process being studied.

However, we must apply trimming carefully. If we incorrectly classify a valid data point as an outlier, we risk discarding genuine information. This can skew our conclusions and even lead to overfitting. **Overfitting** occurs when a model performs remarkably well on the cleaned training data but struggles to make accurate predictions when faced with new data that includes legitimate extreme cases. So, while trimming can simplify analysis, we need to ensure we are not removing valuable or rare insights from our dataset.

```
#Append relevant the CODE of Program 1, 3, 4 and 7 or else this
program will give an error
# Trimming Strategy: Filter the DataFrame to keep only non-
outliers
df_trimmed = df[(df_outlier<= upper_limit_iqr) &
(df_outlier>= lower_limit_iqr)
].copy()

print(f"\nOriginal count: {len(df_outlier)}, Trimmed count:
{len(df_trimmed)}")
print("\nTrimmedDataFrame Head:")
print(df_trimmed[['Customer_ID', 'Age', 'Age_Group',
'Annual_Income', 'Income_Quartile']])
```

Program 8: Trimming outliers

Output:

Original count: 20, Trimmed count: 17

TrimmedDataFrame Head:

	Customer_ID	Age	Age_Group	Annual_Income	Income_Quartile
0	1	18	Youth	15000	Low
1	2	22	Youth	18000	Low
2	3	25	Youth	25000	Low
3	4	27	Adult	27000	Low
4	5	30	Adult	29000	Low
5	6	32	Adult	31000	Medium
6	7	35	Adult	33000	Medium
7	8	38	Adult	35000	Medium
8	9	40	Adult	38000	Medium
9	10	42	Middle-Aged	40000	Medium
10	11	45	Middle-Aged	42000	High
11	12	50	Middle-Aged	45000	High
12	13	52	Middle-Aged	47000	High
13	14	55	Middle-Aged	49000	High
14	15	58	Middle-Aged	50000	High
15	16	60	Middle-Aged	55000	Very High
16	17	62	Senior	70000	Very High

Capping (Winsorization)

Capping, also known as **Winsorization**, offers a gentler alternative. Instead of deleting the extreme values altogether, this approach replaces them with less extreme values at predefined threshold limits. For instance, any value above the upper threshold is set equal to that threshold, and any value below the lower threshold is replaced with the lower bound.

This method helps us maintain the same number of observations in our dataset, which means we avoid losing potential information. At the same time, capping reduces the disproportionate influence of extreme values on our overall distribution. You may prefer this approach when preserving as much data as possible is important or when the outliers, though large, could still represent plausible variations in your population.

```
#Append relevant the CODE of Program 1, 3, 4 and 7 or else this
program will give an error
# Apply Capping (Winsorization) using NumPy's np.where
# Values above the upper limit are replaced by the upper limit,
and vice versa.
df['Annual_Income_Z_Capped'] = np.where(
df_outlier>upper_limit_z,
upper_limit_z,
np.where(df_outlier<lower_limit_z,
lower_limit_z,
df_outlier
)
)

print("\nStatistics before and after Z-Score Capping:")
print(df[['Customer_ID', 'Age', 'Age_Group', 'Income_Quartile',
'Annual_Income_Z_Capped']])
```

Program 9: Apply Capping (Winsorization)

In this program, please note the computation of *Annual_Income_Z_Capped*. This computation requires the use of Numpy library. Wherever the *Annual_Income* is either higher or lower than the higher or lower limiting z values, *Annual_Income_Z_Capped* is replaced by that corresponding value.

Output:

	Customer_ID	Age	Age_Group	Income_Quartile	Annual_Income_Z_Capped
0	1	18	Youth	Low	15000.00000
1	2	22	Youth	Low	18000.00000
2	3	25	Youth	Low	25000.00000
3	4	27	Adult	Low	27000.00000
4	5	30	Adult	Low	29000.00000
5	6	32	Adult	Medium	31000.00000
6	7	35	Adult	Medium	33000.00000
7	8	38	Adult	Medium	35000.00000
8	9	40	Adult	Medium	38000.00000
9	10	42	Middle-Aged	Medium	40000.00000
10	11	45	Middle-Aged	High	42000.00000
11	12	50	Middle-Aged	High	45000.00000
12	13	52	Middle-Aged	High	47000.00000
13	14	55	Middle-Aged	High	49000.00000
14	15	58	Middle-Aged	High	50000.00000
15	16	60	Middle-Aged	Very High	55000.00000
16	17	62	Senior	Very High	70000.00000
17	18	65	Senior	Very High	85000.00000
18	19	68	Senior	Very High	120000.00000
19	20	70	Senior	Very High	179543.41777

In summary, trimming and capping both serve as valuable tools for managing outliers. As analysts, our goal is to choose the method that best maintains the integrity of our dataset while ensuring our models remain robust and realistic.

Advanced Consideration: Trade-offs in Outlier Treatment

When we reach the stage of deciding how to treat outliers, we face an important design decision that shapes the quality of our data preparation. Choosing between trimming and capping is not simply a technical adjustment, it's a strategic choice about how much information we are willing to modify or discard. Trimming may offer greater statistical stability by removing noisy extremes, but at the cost of losing potentially useful, context-rich insights. On the other hand, capping allows us to retain all observations and maintain relationships among variables; however, it can lead to artificial clustering at the boundary values where the data have been capped.

In situations where our data are strictly positive and show strong right-skewness, such as income levels or property prices, another useful approach is the logarithmic

transformation. By applying a logarithmic scale, we compress the range of extreme values, pulling unusually large observations closer to the mean. This adjustment helps reduce their undue influence on the model's variance, making the data more balanced before further processing, such as binning or predictive modeling.

Ultimately, every decision about outlier handling should be guided by sound domain knowledge. We need to understand what the data represent and whether the extremes reflect true phenomena or mere errors. For example, if an outlier corresponds to a known, one-time incident, say, a machine malfunction in a production line, then removing it through trimming can be fully justified. However, when the outlier reflects an authentic but rare event, such as an unusually high yet genuine income or property value, capping or transforming the data is usually a wiser approach. This way, we maintain both model reliability and the authenticity of the real-world patterns our data describe.

b. Advanced Consideration: Logarithmic Transformation

For data that is highly skewed and strictly positive (like certain financial metrics), a common alternative is to apply a Logarithmic Transformation ($\log(X)$). This is not strictly capping, but it mathematically compresses the large variance caused by extreme high values, pulling them closer to the mean and making the distribution more symmetrical, often eliminating the need for hard capping.

Check Your Progress - 2

1. A feature has $Q1 = 150$ and $Q3 = 250$. Calculate the Interquartile Range (IQR) and the upper fence boundary for outlier detection.

.....

2. If you have a very small dataset ($N=50$) with 5% outliers, would you generally prefer trimming (removal) or capping? Justify your answer.

.....

3. What specific type of distribution is the Z-score method best suited for, and why is it problematic to use it directly on highly skewed data?

.....

3.4 PERMUTATION AND RANDOM SAMPLING

So far, we have discussed about the methods with small size of data. However, in the present time, you get large data sets. This section suggests ways to deal with the large data sets.

The Need for Sampling and Permutation

When working with very large datasets, training a model on the full population can be computationally slow or even impossible. Random Sampling is the technique of extracting a smaller, representative subset of data.

Permutation, or shuffling, is the process of randomly rearranging the order of observations. This is crucial before splitting data for model training to ensure that any underlying structure or order in the collected data does not bias our test results.

3.4.1 Random Sampling without Replacement

The simplest form of sampling is selecting rows **without replacement** (i.e., once a row is selected, it cannot be selected again). This is the standard procedure for creating unbiased training and testing datasets. We use `df.sample()` in pandas for sampling. The `random_state` parameter is crucial for educational and professional reproducibility. Setting it to any specific integer ensures that the resulting random selection is the same every time the code is executed.

Creating Sample Datasets

To demonstrate sampling, we will use a small, synthetic dataset representing customer purchase behaviour. This dataset is used in Program 10.

```
import pandas as pd
import numpy as np

# Sample dataset
data = {
    'Customer_ID': range(1, 11),
    'Gender': ['Male', 'Female', 'Female', 'Male', 'Female',
              'Male', 'Female', 'Male', 'Male', 'Female'],
    'Age': [22, 25, 28, 21, 35, 45, 32, 26, 40, 23],
    'Amount': [120, 340, 560, 90, 300, 800, 650, 200, 400,
               150],
    'Category': ['Clothing', 'Electronics', 'Groceries',
                 'Clothing', 'Groceries', 'Electronics',
                 'Clothing', 'Groceries', 'Electronics',
                 'Electronics', 'Clothing']
}

df = pd.DataFrame(data)
print("---- Original Data ----")
print(df)
```

Program 10: Sample Data Set of Customer Purchases

Output:

```
--- Original Data ---
  Customer_ID  Gender  Age  Amount  Category
0           1    Male   22    120   Clothing
1           2  Female   25    340 Electronics
2           3  Female   28    560   Groceries
3           4    Male   21     90   Clothing
4           5  Female   35    300   Groceries
5           6    Male   45    800 Electronics
6           7  Female   32    650   Clothing
7           8    Male   26    200   Groceries
8           9    Male   40    400 Electronics
9          10  Female   23    150   Clothing
```

Let us try to select data randomly. Using the parameter `random_state` ensures that if you run the code again, you get the exact same sample, which is vital for reproducibility.

```
# Add code of Program 10 here.
# Shuffle the dataset randomly without replacement
# 1. Sampling a fixed number of rows (n=10)
df_permuted = df.sample(n=10, random_state=42)
print(f"Sampled 10 rows. Size: {len(df_permuted)}")
print("Sampled rows indices (n=10):",
df_permuted.index.tolist())

print("--- Original Order ---")
print(df[['Customer_ID', 'Amount']])

print("\n--- Permuted Order without replacement ---")
print(df_permuted[['Customer_ID', 'Amount']])
```

Program 11: Subsetting Data (fixed number of rows n=10)**Output:**

```
Sampled 10 rows. Size: 10
Sampled rows indices (n=10): [8, 1, 5, 0, 7, 2, 9, 4, 3, 6]
--- Original Order ---
  Customer_ID  Amount
0           1     120
1           2     340
2           3     560
3           4      90
4           5     300
5           6     800
6           7     650
7           8     200
8           9     400
9          10     150

--- Permuted Order without replacement ---
  Customer_ID  Amount
8           9     400
1           2     340
5           6     800
0           1     120
7           8     200
2           3     560
9          10     150
4           5     300
3           4      90
6           7     650
```

Sampling and Subsetting Data

Sampling data using `frac` allows fractions of the data. Setting `frac=1` returns all rows that are shuffled.

Let us try to select 60% of our data randomly. Using the parameter `random_state` ensures that if you run the code again, you get the exact same sample, which is vital for reproducibility.

```
# Add code of Program 10 here
# 2. Sampling a fraction of the data (frac=60%)
df_permuted_frac = df.sample(frac=0.60, random_state=42)
print(f"Sampled 60% of data. Size: {len(df_permuted_frac)}")

print("--- Permuted Order 60% data ---")
print(df_permuted_frac[['Customer_ID', 'Amount']])
```

Program 12: Sample Data (60%) with same random_state

Since we have kept the value of `random_state` in Program 10 and Program 11 the same (42), we have got the same permutation sequence in both outputs, except that in the output of Program 11 only 6 records out of 10 got sampled, as we have used `frac=0.6`.

Output:

```

Sampled 60% of data. Size: 6
--- Permuted Order 60% data ---
  Customer_ID  Amount
8           9     400
1           2     340
5           6     800
0           1     120
7           8     200
2           3     560

```

3.4.2 Random Sampling with Replacement (Bootstrapping)

If you set the parameter `replace=True`, the same row can be selected multiple times in the resulting sample. This is the core mechanic used in:

- **Bootstrapping:** A statistical method where multiple resamples are taken (with replacement) to estimate the variability of a population parameter.
- **Upsampling:** Used in machine learning to balance imbalanced datasets by over-representing the minority class.

The parameter `replace=True` allows the same row to be selected multiple times, a feature utilised in advanced statistical methods. Conversely, `replace=False` (the default) ensures unique selections and is necessary when selecting training or test sets.

Bootstrapping

Sampling data with replacement. Setting `replace=True` returns shuffled data with replacement.

```

# Add code of Program 10 here
# 3. Sampling with replacement (useful for bootstrapping)
df_bootstrap = df.sample(n=6, replace=True, random_state=42)
print(f"Sampled 6 rows with replacement (replace=True). Size: {len(df_bootstrap)}")

print("---- Permuted Order with replacement ----")
print(df_bootstrap[['Customer_ID', 'Amount']])

```

Program 13: Subsetting Data (with replacement - Bootstrapping)

You may please check that Bootstrapping may result in selection of a data row more than once in a sample. For example, in the following output data `Customer_ID 7` is selected two times.

Output:

```

Sampled 6 rows with replacement (replace=True). Size: 6
--- Permuted Order with replacement ---
  Customer_ID  Amount
6           7     650
3           4      90
7           8     200
4           5     300
6           7     650
9          10     150

```

Upsampling

If we set `frac=1.2` (120% of data) along with `replace=True`, we create a larger sample containing duplicates.

```
# Add code of Program 10 here
# Select 120% (frac=1.2) of the data, allowing repeats
(replace=True)
df_upsampled = df.sample(frac=1.2, replace=True,
random_state=1)

print("--- Original Data ---")
print(df)

print(f"\nUpsampled Data Size: {len(df_upsampled)}")
print("--- Upsampled (Note potential duplicates) ---")
print(df_upsampled)
```

Program 14: Upsampling Data**Output:**

```
--- Original Data ---
Customer_ID Gender Age Amount Category
0 1 Male 22 120 Clothing
1 2 Female 25 340 Electronics
2 3 Female 28 560 Groceries
3 4 Male 21 90 Clothing
4 5 Female 35 300 Groceries
5 6 Male 45 800 Electronics
6 7 Female 32 650 Clothing
7 8 Male 26 200 Groceries
8 9 Male 40 400 Electronics
9 10 Female 23 150 Clothing

Upsampled Data Size: 12
--- Upsampled (Note potential duplicates) ---
Customer_ID Gender Age Amount Category
5 6 Male 45 800 Electronics
8 9 Male 40 400 Electronics
9 10 Female 23 150 Clothing
5 6 Male 45 800 Electronics
0 1 Male 22 120 Clothing
0 1 Male 22 120 Clothing
1 2 Female 25 340 Electronics
7 8 Male 26 200 Groceries
6 7 Female 32 650 Clothing
9 10 Female 23 150 Clothing
2 3 Female 28 560 Groceries
4 5 Female 35 300 Groceries
```

3.4.3 Permutation (Shuffling Data Order)

Permutation refers to the random rearrangement of elements in a sequence. In Data Science, we often need to shuffle the rows of a dataset before splitting it into training and testing sets to eliminate any ordering bias that might exist (e.g., if customer data was sorted alphabetically or chronologically).

The NumPy library provides the `np.random.permutation()` function, which takes an array-like object and returns a new array with the elements randomly shuffled, crucially leaving the original array unchanged.

```
arr = np.array(range(10))
print(f"Original array: {arr}")

# Generate a new, randomly permuted array
shuffled_arr = np.random.permutation(arr)
print(f"Permuted array: {shuffled_arr}")

# Verify that the original array remains unchanged
print(f"Original array remains: {arr}")
```

Program 15: Shuffling Indices using NumPy

Output:

```
Original array: [0 1 2 3 4 5 6 7 8 9]
Permuted array: [3 5 7 0 2 4 6 8 9 1]
Original array remains: [0 1 2 3 4 5 6 7 8 9]
```

Shuffling a DataFrame's Rows

To shuffle a Pandas DataFrame efficiently, we generate a random permutation of its indices and then use those new indices to reorder the rows. This ensures that the data integrity of each row (i.e., the values within a single observation) is maintained while the overall arrangement is randomised.

```
# Add code of Program 10 here
# Generate a permutation of the indices (0, 1, 2, 3, 4)
shuffled_indices = np.random.permutation(len(df))

print(f"\nShuffled indices: {shuffled_indices}")

# Use .iloc to reorder the DataFrame based on the new indices
df_shuffled = df.iloc[shuffled_indices]

print("\n--- Shuffled DataFrame ---")
print(df_shuffled)
```

Program 16: Shuffling rows of a dataframe**Output:**

```
Shuffled indices: [3 4 5 6 0 1 7 2 9 8]

--- Shuffled DataFrame ---
   Customer_ID  Gender  Age  Amount  Category
3            4    Male   21     90   Clothing
4            5  Female   35    300  Groceries
5            6    Male   45    800 Electronics
6            7  Female   32    650   Clothing
0            1    Male   22    120   Clothing
1            2  Female   25    340 Electronics
7            8    Male   26    200  Groceries
2            3  Female   28    560  Groceries
9           10  Female   23    150   Clothing
8            9    Male   40    400 Electronics
```

Check Your Progress - 3

1. If you use `df.sample(n=100, replace=True)`, what type of sampling is being performed, and for what statistical technique is this commonly used?

.....

.....

2. You are splitting a 10,000-row dataset into training and testing sets (80/20 split). Write the Python command using `df.sample()` that selects the 80% training set and ensures that the selection is the same every time you run the script.

.....

.....

3. When shuffling a 2D NumPy array using `np.random.permutation()`, what dimension (axis) is permuted by default?

.....

.....

.....

3.5 COMPUTING INDICATOR/DUMMY VARIABLES

In this section, we discuss the use of dummy variables in the context of data transformation.

3.5.1 The Challenge of Categorical Data

Most sophisticated machine learning models rely on mathematical calculations and require numerical input. When we encounter **Nominal Categorical Features** (categories without a specific order, like Department: Finance, IT, Sales), we cannot simply assign integers (1, 2, 3). If we did, the model would incorrectly assume that Sales (3) is "greater" than IT (2), which is meaningless.

The solution is **One-Hot Encoding (OHE)**, which converts these categories into a numerical format using binary (0 or 1) columns, known as **Dummy** or **Indicator Variables**.

3.5.2 The Principle of One-Hot Encoding

One-Hot Encoding takes a column with k unique categories and creates k new binary (0 or 1) columns. For any given row, the corresponding category column is marked as 1 (or True) and all other category columns are marked as 0 (or False). This preserves the categorical information without implying any false sense of numerical order or magnitude between the categories.

The pandas library simplifies this operation dramatically using the `pd.get_dummies()` function.

```
import pandas as pd
import numpy as np

# Sample dataset
data = {
    'Customer_ID': range(1, 11),
    'Gender': ['Male', 'Female', 'Female', 'Male', 'Female',
              'Male', 'Female', 'Male', 'Male', 'Female'],
    'Age': [22, 25, 28, 21, 35, 45, 32, 26, 40, 23],
    'Amount': [120, 340, 560, 90, 300, 800, 650, 200, 400,
               150],
    'Category': ['Clothing', 'Electronics', 'Groceries',
                 'Clothing',
                 'Groceries', 'Electronics', 'Clothing',
                 'Groceries',
                 'Electronics', 'Clothing']
}

df = pd.DataFrame(data)
print("--- Original Data ---")
print(df)

# Convert Gender and Product_Category to dummy variables
df_dummies = pd.get_dummies(df, columns=['Gender', 'Category'],
                             drop_first=False)
print("\n--- Data after One-Hot Encoding ---")
print(df_dummies)
```

Program 17: Basic Dummy Variable Creation

Output:

```

--- Original Data ---
Customer_ID  Gender  Age  Amount  Category
0            1    Male   22    120    Clothing
1            2  Female   25    340  Electronics
2            3  Female   28    560  Groceries
3            4    Male   21     90    Clothing
4            5  Female   35    300  Groceries
5            6    Male   45    800  Electronics
6            7  Female   32    650    Clothing
7            8    Male   26    200  Groceries
8            9    Male   40    400  Electronics
9           10  Female   23    150    Clothing

--- Data after One-Hot Encoding ---
Customer_ID  Age  Amount  Gender_Female  Gender_Male  Category_Clothing \
0            1   22    120          False         True         True
1            2   25    340          True          False        False
2            3   28    560          True          False        False
3            4   21     90          False         True         True
4            5   35    300          True          False        False
5            6   45    800          False         True         False
6            7   32    650          True          False         True
7            8   26    200          False         True         False
8            9   40    400          False         True         False
9           10   23    150          True          False         True

Category_Electronics  Category_Groceries
0                   False                False
1                   True                 False
2                   False                True
3                   False                False
4                   False                True
5                   True                 False
6                   False                False
7                   False                True
8                   True                 False
9                   False                False

```

As you can see, for each row, only one column has a value of *True*, indicating the presence of that specific Category.

Avoiding the Dummy Variable Trap ($k-1$ Encoding)

A critical consideration when using One-Hot Encoding in statistical modeling (particularly linear regression) is the potential for **perfect multicollinearity**, known as the **Dummy Variable Trap**. If a categorical variable has k distinct levels, and we create k dummy variables, the information in the k th variable is completely redundant because it can be perfectly inferred from the other $k-1$ variables.

This redundancy can cause matrix singularity and instability in many linear models. To resolve this, standard practice is to use $k-1$ encoding, where one category's dummy variable is intentionally dropped, typically the first one. We achieve this using the parameter `drop_first=True`. The dropped category becomes the baseline, represented by a row of all zeros in the remaining indicator columns.

```

# One-Hot Encoding with Drop First

# Drop the first category encountered Gender('Female') and
Category('Clothing')
dummies_df_reduced = pd.get_dummies(df, columns=['Gender',
'Category'], drop_first=True)

print("--- Dummy Variables (Drop First): ---")
print(dummies_df_reduced)

```

Program 18: Implementing $k-1$ Encoding

Output:

```

--- Dummy Variables (Drop First): ---
  Customer_ID  Age  Amount  Gender_Male  Category_Electronics \
0            1   22    120         True         False
1            2   25    340         False         True
2            3   28    560         False         False
3            4   21     90         True         False
4            5   35    300         False         False
5            6   45    800         True          True
6            7   32    650         False         False
7            8   26    200         True         False
8            9   40    400         True          True
9           10   23    150         False         False

  Category_Groceries
0             False
1             False
2              True
3             False
4              True
5             False
6             False
7              True
8             False
9             False

```

Check Your Progress - 4

1. A column has the categories 'North', 'South', 'East', 'West'. If you use `pd.get_dummies()` with `drop_first=True`, how many new columns will be created?

.....

.....

.....

2. Define the “Dummy Variable Trap” and explain why it is problematic for linear models.

.....

.....

.....

3. When would setting the parameter `dummy_na=True` be a desirable feature engineering choice?

.....

.....

.....

4. If a column has 7 unique categories, how many dummy variables should you retain after applying OHE to avoid multicollinearity?

.....

.....

.....

3.6 STRING MANIPULATION AND REGULAR EXPRESSIONS

Text data (strings) is frequently unstructured and inconsistent. Data cleaning and standardisation are critical transformation steps, often involving case changes, whitespace removal, and the extraction of embedded information.

3.6.1 Vectorised String Object Methods in pandas

Text data (strings) in pandas series require specialised handling. You cannot apply standard Python string methods (like `.lower()` or `.strip()`) directly to an entire pandas Series. Instead, you must use the `.str` accessor. The `.str` accessor applies the string operation element-wise across every row, achieving high performance through vectorisation.

Common cleaning tasks that rely on the `.str` accessor include :

1. **Case Transformation:** Using `.str.lower()` or `.str.upper()` to standardise case.
2. **Whitespace Removal:** Using `.str.strip()` to eliminate leading and trailing spaces.
3. **Replacement:** Using `.str.replace(old, new)` to fix simple typos or remove specific characters.
4. **Length Calculation:** Using `.str.len()` to find the length of each string.

Creating Sample Datasets

To demonstrate string manipulation, we will use a small, synthetic dataset simulating customer feedback collected from an online store.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Simulating Customer Feedback
data = {
    'Customer_ID': [101, 102, 103, 104, 105],
    'Name': ['john doe ', 'JANE DOE', 'robert_smith', 'Alice Johnson', 'michael@brown'],
    'Email': ['john.doe@gmail.com', 'jane_doe@yahoo.com', 'robert.smith@outlook.com', 'alice.johnson@example.com', 'michael.brown@company.org'],
    'Feedback': [
        'Loved the Product! Great quality!!!',
        'average service... could be Better',
        'BAD EXPERIENCE!! will not recommend ',
        'Excellent! fast delivery and good packaging',
        'price is high, but quality_good.'
    ],
    'Phone': ['+91-9876543210', '98765 43210', '91-9000090000', '080-12345678', '90000abc900']
}

df = pd.DataFrame(data)
print("--- Original Data ---")
print(df)
```

Program 19: Create Sample Data for String Manipulation

Output:

```

--- Original Data ---
Customer_ID      Name      Email \
0      101      john doe      john.doe@gmail.com
1      102      JANE DOE      jane_doe@yahoo.com
2      103      robert_smith      robert.smith@outlook.com
3      104      Alice Johnson      alice.johnson@example.com
4      105      michael@brown      michael.brown@company.org

Feedback      Phone
0      Loved the Product! Great quality!!!      +91-9876543210
1      average service... could be Better      98765 43210
2      BAD EXPERIENCE!! will not recommend      91-9000090000
3      Excellent! fast delivery and good packaging      080-12345678
4      price is high, but quality_good.      90000abc900

```

Program 20: Basic Cleaning and Case Standardisation

Let us apply basic cleaning such as whitespace, casing, or unwanted characters before analysis.

```

# Add code of Program 19 here
# Trim whitespaces and standardize casing
df['Name_Cleaned'] = df['Name'].str.strip().str.title()

# Lowercase emails
df['Email_Cleaned'] = df['Email'].str.lower()

# Remove extra spaces in feedback and lowercase it
df['Feedback_Cleaned'] = df['Feedback'].str.strip().str.lower()

print("--- Basic String Manipulation ---")
print(df[['Name', 'Name_Cleaned', 'Email_Cleaned',
'Feedback_Cleaned']])

```

Output:

```

--- Basic String Manipulation ---
Name      Name_Cleaned      Email_Cleaned \
0      john doe      John Doe      john.doe@gmail.com
1      JANE DOE      Jane Doe      jane_doe@yahoo.com
2      robert_smith      Robert_Smith      robert.smith@outlook.com
3      Alice Johnson      Alice Johnson      alice.johnson@example.com
4      michael@brown      Michael@Brown      michael.brown@company.org

Feedback_Cleaned
0      loved the product! great quality!!!
1      average service... could be better
2      bad experience!! will not recommend
3      excellent! fast delivery and good packaging
4      price is high, but quality_good.

```

3.6.2 Structural Manipulations and Feature Creation

The `.str` accessor allows for structural transformations that are crucial for feature engineering .

```

# Add code of Program 19 here
# Find if feedback contains the word 'good'
df['Contains_Good'] =
df['Feedback_Cleaned'].str.contains('good')

# Count number of characters in feedback
df['Feedback_Length'] = df['Feedback_Cleaned'].str.len()

# Replace underscores and special chars in names

```

```
df['Name_Normalized'] = df['Name_Cleaned'].str.replace('_', ' ', regex=False)

print("---- String Manipulation Feature Creation ----")
print(df[['Name_Cleaned', 'Name_Normalized', 'Feedback_Length', 'Contains_Good']])
```

Code 21: String Manipulation Feature Creation**Output:**

```
--- String Manipulation Feature Creation ---
   Name_Cleaned Name_Normalized  Feedback_Length  Contains_Good
0      John Doe      John Doe             35          False
1      Jane Doe      Jane Doe             34          False
2  Robert_Smith  Robert Smith             35          False
3  Alice Johnson  Alice Johnson             43           True
4 Michael@Brown Michael@Brown             32           True
```

Introduction to Regular Expressions (Regex)

While simple string methods handle exact replacements and basic cleaning, **Regular Expressions (Regex)** provide the mechanism for matching and manipulating complex, variable, or fuzzy text patterns. They allow you to match, find, and manipulate text even when the data is inconsistent (e.g., finding all numbers, regardless of how many spaces surround them). Regex uses a compact syntax of special characters (e.g., quantifiers like `*` and `+`, and character classes like `\d` for digits) to define dynamic search criteria.

In Python data transformation, Regex is typically accessed via the standard `re` module, or more frequently, integrated directly into Pandas through the `.str` accessor, allowing functions like `str.replace()` and `str.contains()` to accept Regex patterns.

Regex Pattern	Description	Example Match
<code>\d</code>	Matches any single digit (0–9).	<code>\d+</code> matches 123, 45
<code>\w</code>	Matches any word character (alphanumeric, underscore).	<code>\w</code> matches 'A', '1', '_'
<code>\s</code>	Matches any whitespace character (space, tab, newline).	<code>\s+</code> matches multiple spaces
<code>.</code>	Matches any character (except newline).	
<code>^ / \$</code>	Anchors the pattern to the start / end of the string.	<code>^abc</code> matches only strings starting with 'abc'
<code>[a-z]</code>	Matches any character in the defined set (Character Set).	<code>[0-9]</code> matches any digit
<code>+ / * / {N}</code>	Quantifiers: one or more / zero or more / exactly \$N\$ times.	<code>\d{4}</code> matches exactly four digits
<code>[^\w\s]</code>	Negated Set: Matches anything NOT a word character or whitespace.	Used to remove punctuation

Cleaning using Regex Replacement

The `.str.replace()` method, combined with Regex, is ideal for standardising text by removing unwanted characters or spaces.

Let us clean a simulated column of raw, user-generated text feedback.

```
# Add code of Program 19 here
# Regex Cleaning - Removing Punctuation and Standardizing Spacing

# Step 1: Remove all non-word or non-whitespace characters
([^\w\s])
```

```
df['Feedback_Cleaned_punc'] =
df['Feedback_Cleaned'].str.replace(r'^\w\s', '', regex=True)

# Step 2: Replace multiple spaces (\s+ matches one or more
spaces) with a single space (' ')
df['Feedback_Cleaned_final'] =
df['Feedback_Cleaned_punc'].str.replace(r'\s+', ' ',
regex=True).str.strip()

# Count number of characters in feedback
df['Feedback_Length'] = df['Feedback_Cleaned'].str.len()

print("--- Feedback Transformed ---")
print(df[['Feedback', 'Feedback_Cleaned', 'Feedback_Cleaned_punc',
'Feedback_Cleaned_final']])
```

Program 22: Removing Punctuation and Standardising Spacing

Output:

```
--- Feedback Transformed ---
                                Feedback \
0      Loved the Product! Great quality!!!
1      average service... could be Better
2      BAD EXPERIENCE!! will not recommend
3      Excellent! fast delivery and good packaging
4      price is high, but quality_good.

                                Feedback_Cleaned \
0      loved the product! great quality!!!
1      average service... could be better
2      bad experience!! will not recommend
3      excellent! fast delivery and good packaging
4      price is high, but quality_good.

                                Feedback_Cleaned_punc \
0      loved the product great quality
1      average service could be better
2      bad experience will not recommend
3      excellent fast delivery and good packaging
4      price is high but quality_good

                                Feedback_Cleaned_final
0      loved the product great quality
1      average service could be better
2      bad experience will not recommend
3      excellent fast delivery and good packaging
4      price is high but quality_good
```

Extraction with Regex

One of the most powerful uses of Regular Expressions in data transformation is to extract structured features from unstructured text. This is often achieved using capture groups, defined by placing the desired pattern inside parentheses () within the overall Regex string. The Pandas method `str.extract()` is specifically designed to isolate these capture groups and place them into new columns.

```
# Add code of Program 19 here
# Extract email domain using regex
df['Email_Domain'] = df['Email_Cleaned'].str.extract(r'@([A-Za-z0-9.-]+)')

# Extract only digits from phone numbers
df['Phone_Digits'] = df['Phone'].str.replace(r'\D', '',
regex=True)

# Validate phone number (should be 10 digits)
df['Valid_Phone'] = df['Phone_Digits'].str.match(r'^\d{10}$')
```

```
# Detect invalid emails (contains multiple '@')
df['Valid_Email'] = ~df['Email_Cleaned'].str.contains(r'@{2,}')

print("---- Regex Manipulation Feature Creation ----")
print(df[['Email', 'Email_Domain', 'Phone', 'Phone_Digits',
'Valid_Phone', 'Valid_Email', 'Feedback_Cleaned_final']])
```

Program 23: Extracting Domains from email**Output:**

```
--- Regex Manipulation Feature Creation ---
      Email  Email_Domain  Phone  Phone_Digits  \
0  john.doe@gmail.com    gmail.com  +91-9876543210  919876543210
1  jane_doe@yahoo.com    yahoo.com    98765 43210    9876543210
2  robert.smith@outlook.com  outlook.com  91-9000090000  919000090000
3  alice.johnson@example.com  example.com  080-12345678  08012345678
4  michael.brown@company.org  company.org    90000abc900    90000900

      Valid_Phone  Valid_Email  Feedback_Cleaned_final
0          False         True    loved the product great quality
1           True         True    average service could be better
2          False         True    bad experience will not recommend
3          False         False  excellent fast delivery and good packaging
4          False         True    price is high but quality_good
```

```
# Add code of Program 19 here
# Custom function to categorize sentiment based on keywords
def sentiment_category(text):
    text = text.lower()
    if 'bad' in text or 'not recommend' in text:
        return 'Negative'
    elif 'good' in text or 'loved' in text or 'excellent' in
text:
        return 'Positive'
    else:
        return 'Neutral'

df['Sentiment'] =
df['Feedback_Cleaned'].apply(sentiment_category)
print("---- Sentiment Analysis ----")
print(df[['Feedback_Cleaned', 'Sentiment']])

df_final = df[[
    'Customer_ID', 'Name_Normalized', 'Email_Domain',
    'Phone_Digits', 'Valid_Phone', 'Valid_Email',
    'Feedback_Cleaned_final', 'Sentiment'
]]
print("\n--- Final Sentiment Analysis ---")
print(df_final)
```

Program 24: Sentiment Analysis**Output:**

```
--- Sentiment Analysis ---
      Feedback_Cleaned  Sentiment
0    loved the product! great quality!!!  Positive
1    average service... could be better    Neutral
2    bad experience!! will not recommend  Negative
3  excellent! fast delivery and good packaging  Positive
4    price is high, but quality_good.  Positive

--- Final Sentiment Analysis ---
      Customer_ID  Name_Normalized  Email_Domain  Phone_Digits  Valid_Phone  \
0             101         John Doe    gmail.com  919876543210         False
1             102         Jane Doe    yahoo.com   9876543210          True
2             103    Robert Smith  outlook.com  919000090000         False
3             104    Alice Johnson  example.com  08012345678         False
4             105  Michael@Brown  company.org    90000900         False

      Valid_Email  Feedback_Cleaned_final  Sentiment
0             101         John Doe    gmail.com  919876543210         False
1             102         Jane Doe    yahoo.com   9876543210          True
2             103    Robert Smith  outlook.com  919000090000         False
3             104    Alice Johnson  example.com  08012345678         False
4             105  Michael@Brown  company.org    90000900         False
```

0	True	loved the product great quality	Positive
1	True	average service could be better	Neutral
2	True	bad experience will not recommend	Negative
3	False	excellent fast delivery and good packaging	Positive
4	True	price is high but quality_good	Positive

3.7 SUMMARY

This unit explains details about the processes of data transformation. First, the unit describes the process of data discretisation that transforms continuous numeric data to discrete categories. The unit explains the concept of equal-width and equal-frequency binning with the help of Python example. It also compares the Python functions that are used for the binning process.

Next, the unit discusses the concept of detecting and filtering probable outliers. It explains in details how probable outlier can be identified by using boxplot or z-score methods. It then explains the process of filtering the possible outliers using trimming and capping methods.

The unit also explains the process of permutation to perform random sampling to select representative data sample using subsetting and bootstrapping. This process helps in removing the bias of data due ordering of data in datasets.

This unit also discusses about the dummy variables that can be crated from nominal categorical columns. This process requires that the multicollinearity problem does not appear in such data.

Finally, the unit describes the process of using string objects and regular expressions.

The entire unit is using Python pandas library for demonstrating the examples of these concepts. However, the concepts of data transformation are changing rapidly, therefore, you must keep studying about the latest developments in the area.

3.8 SOLUTIONS/ANSWERS

Check Your Progress 1

1. **Equal-Frequency Binning** (`pd.qcut`). This method is preferred because the goal is equal representation. `pd.qcut` uses quantiles to ensure that regardless of the data's skewness, approximately the same count of customers is placed in each income bin.
2. The primary advantage is to **reduce the training complexity and computation time**. Discretisation reduces the number of unique points where the decision tree algorithm must consider potential splits, resulting in a more efficient induction process.
3. The range is $100 - 0 = 100$. With 5 bins, the width of each interval is $100 / 5 = 20$ units.

Check Your Progress 2

1. $IQR = Q3 - Q1 = 250 - 150 = 100$. The upper fence boundary is calculated as $Q3 + 1.5 \times IQR = 250 + (1.5 \times 100) = 400$.
2. You would generally prefer **capping (Winsorization)**. In a small dataset ($N=50$), removing 5% of the data (2 to 3 rows) constitutes a significant loss of information. Capping preserves the sample size while mitigating the impact of the extreme values.

3. The Z-score method is best suited for **Normal (Gaussian) distributions**. It is problematic for highly skewed data because the Mean and Standard Deviation, which define the boundaries, are highly sensitive to extreme outliers. The presence of outliers inflates the standard deviation, making the calculated boundaries too wide and potentially hiding legitimate, less extreme outliers.

Check Your Progress 3

1. **Sampling with replacement**. This statistical technique is fundamental to **bootstrapping**, where samples are drawn repeatedly from the dataset to estimate the accuracy or stability of a statistical estimate or model.
2. `train_set = df.sample(frac=0.8, random_state=101)` (The specific integer used for `random_state` can vary, but its presence ensures reproducibility).
3. `np.random.permutation()` permutes the **rows** (axis 0) of the 2D NumPy array by default.

Check Your Progress 4

1. Since there are four categories ($k=4$), and we use $k-1$ encoding via `drop_first=True`, three new columns will be created (e.g., 'South', 'East', 'West').
2. The **Dummy Variable Trap** is the creation of redundant indicator variables when performing One-Hot Encoding. If a feature has k levels, using k indicator variables means that the k -th variable is perfectly correlated with the others. This perfect redundancy leads to multicollinearity, which violates assumptions in many linear models and causes instability.
3. `dummy_na=True` is a desirable feature engineering choice when the missingness of a value is non-random and holds predictive power. For example, if a `User_Activity` score is NaN only for newly registered users, the NaN value indicates a specific customer segment that should be modeled separately.
4. You need just 6 dummy variables, as 7th variable is redundant.

3.9 FURTHER READING AND REFERENCES

1. McKinney, W. (2017). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. O'Reilly Media.
2. VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.
3. Brownlee, J. (2020). *Data Preparation for Machine Learning: Extracted from the Machine Learning Mastery Library*. Machine Learning Mastery.
4. Pandas Development Team. `pandas.DataFrame.sample`. Official Documentation.
5. NumPy Development Team. `numpy.random.permutation`. Official Documentation.
6. Analytics Vidhya. (2021). *Feature Engineering: How to Detect and Remove Outliers with Python Code*. (Online resource providing practical examples for Z-score and IQR methods).